Kathara Lab

Kathara is a **network emulation tool**. Within the Kathara environment each network node is implemented by a docker container, and each interconnection link is emulated by using a virtual network. Each device can be configured to have an arbitrary number of (virtual) ethernet network interfaces. Each node can be seen as an independent linux machine, for you to configure from scratch.

Create a Kathara Laboraty

First of all, we must create a configuration file.

```
francesco@PCFrancesco:~/Desktop$ mkdir lab_test
francesco@PCFrancesco:~/Desktop$ cd lab_test
francesco@PCFrancesco:~/Desktop/lab_test$ nano lab.conf
```

Here we specify three things:

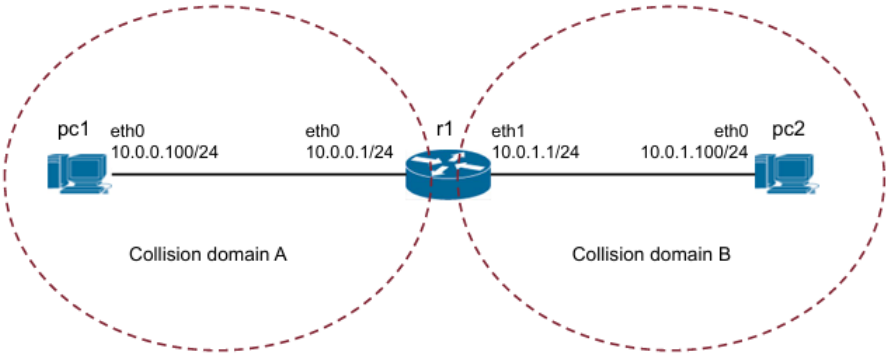
- **node names** (must be unique)
- **network interfaces** of the node, in the form of eth0, eth1, …
- **collision domains** of the interfaces

```
pc1[0]=A #set eth0 of pc1 to collision domain A
r1[0]=A  #set eth0 of r1 to collision domain A

r1[1]=B  #set eth1 of r1 to collision domain B
pc2[0]=B #set eth0 of pc2 to collision domain B
```

NOTE: There can not be spaces " "

The collision domain is a sort of LAN. All the nodes in the same collision domain will receive the same packets.



Now, we've correctly configured kithara. We can start configuring the network using the command:

- **kathara lstart**: to start the laboratory configured in the working directory
- **kathara lclean**: to stop the laboratory

Once started the laboratory kathara will open a command prompt for each node configured in the configuration file.

Commands

Let's see the most common commands.

Ip link

The ip link command let us show and manage (setting up and down) network interfaces:

```
ip link show          # shows the details of every available iface
ip link set eth0 up   # brings up eth0 interface
ip link set eth0 down  # brings down eth0 iface
```

NOTE: By default, the interfaces are set up (s we don't need to set them up on every new lab)

Ip address

Now that the interfaces are up, we are can assign IPv4 addresses to them. The command to assigning IPv4 addresses:

```
ip address show  # show information about addresses
                 # assigned to interfaces

ip address add a.b.c.d/m dev ethX
# assigns a.b.c.d address with
# submask m to device ethX
```

NOTE: When we will work with many nodes will be difficult to remember all the ip addreses. So, we can "memorize" them as alias name:

echo "10.0.1.100 pc2" >> /etc/hosts

This command appends the IP address (10.0.1.100) and its corresponding alias (pc2) to the /etc/hosts file.

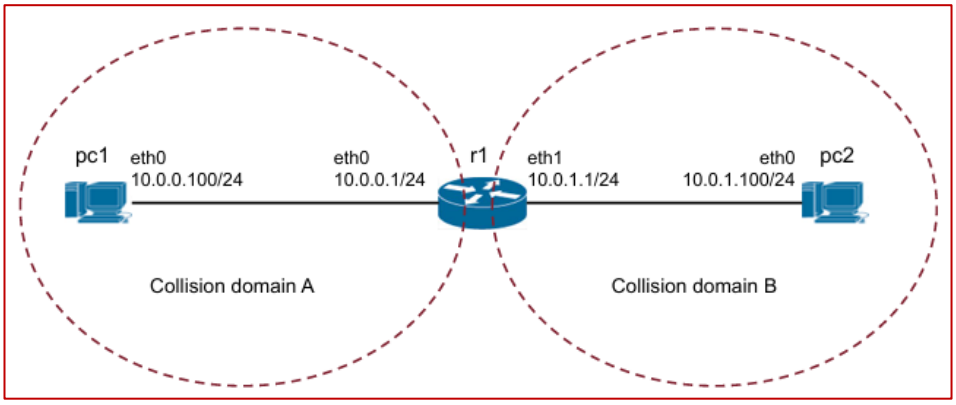
Ping

We can test connection between two hosts using the ping command:

```
ping a.b.c.d      # sends ICMP packets to a.b.c.d
                  # waits for reply
```

It gives us information about Round Trip Time and Time To Live (useful for traceroute) for a connection between 2 hosts.

Exercise: Configure kathara (via *lab.*conf) and use *ip address add* to configure the following network. Then use *ping* from pc1 to r1 and from pc2 to r1:



Solution:

```
francesco@PCFrancesco:~/Desktop$ mkdir lab_test
francesco@PCFrancesco:~/Desktop$ cd lab_test
francesco@PCFrancesco:~/Desktop/lab_test$ nano lab.conf
```

```
pc1[0]=A #set eth0 of pc1 to collision domain A
r1[0]=A  #set eth0 of r1 to collision domain A

r1[1]=B  #set eth1 of r1 to collision domain B
pc2[0]=B #set eth0 of pc2 to collision domain B
```

```
(base) PS C:\Users\39324\Desktop\lab_ex1> kathara lstart
                    Starting Network Scenario
[Deploying collision domains]                                    2/2
[Deploying devices]                                              3/3
```

Network Configuration:

```
root@pc1:/# ip address add 10.0.0.100/24 dev eth0
root@pc1:/#
```

```
root@r1:/# ip address add 10.0.0.1/24 dev eth0
root@r1:/# ip address add 10.0.1.1/24 dev eth1
root@r1:/#
```

```
root@pc2:/# ip address add 10.0.1.100/24 dev eth0
root@pc2:/#
```

Ping pc1 -> r1:

```
root@pc1:/# ping 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=1.19 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.635 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=1.16 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.745 ms
```

Ping pc2 -> r1:

```
PING 10.0.1.1 (10.0.1.1) 56(84) bytes of data.
64 bytes from 10.0.1.1: icmp_seq=1 ttl=64 time=1.10 ms
64 bytes from 10.0.1.1: icmp_seq=2 ttl=64 time=0.976 ms
64 bytes from 10.0.1.1: icmp_seq=3 ttl=64 time=0.831 ms
64 bytes from 10.0.1.1: icmp_seq=4 ttl=64 time=0.362 ms
```

NOTE: If we try to ping pc1 to pc2 we'll get an error.

```
root@pc1:/# ping 10.0.1.100
connect: Network is unreachable
```

This happens because pc1 and pc2 are in two different collision domains and they don't know how to reach each other. To solve this problem, we must use *ip route add* (see below)

Static routes

Let *pc1* and *pc2* be two different nodes on different collision domains. If we try to ping pc1 → pc2, we get "**network unreachable**". But, why? Because pc1 does not know how to reach pc2, because he only has the route to his subnet (collision domain).

Solution: In order to let pc1 and pc2 communicate, we have two ways:

- **Define the "default" route (gateway)**
- **Specify how to reach pc2 subnet from pc1 and vice versa**

The difference between the two approaches will be clearer on bigger topology, but basically by defining a gateway we tell to the host "Give all the packets that are not for your subnet to the gateway, he knows how to route them". In our case the gateway is r1.

With the second approach, we tell to the host "Give all the packets that are for a specific subnet to a specific host (r1), he knows how to route them"

Ip route

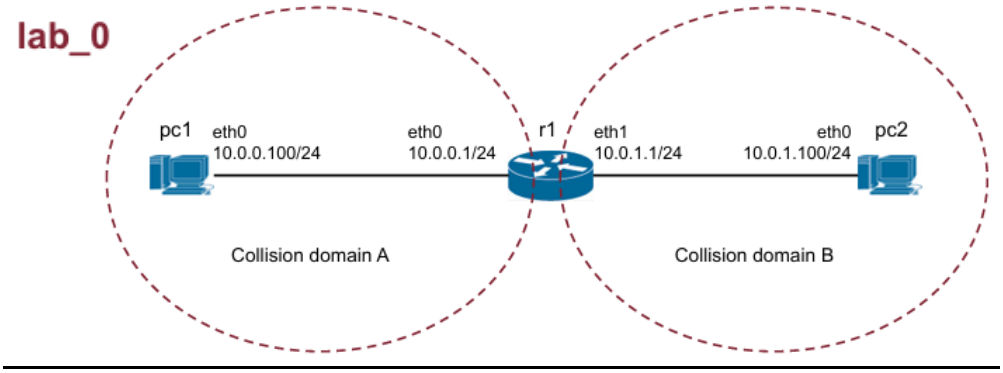
In order to manage routes, we use the command:

```
ip route show    # show the available routes on the host

ip route add a.b.c.d/m via next_hop_ip
# adds the route to subnet a.b.c.d/m via the next_hop_ip

ip route add default via next_hop_ip
ip route add 0.0.0.0/0 via next_hop_ip
# adds the default gateway via the next_hop_ip
```

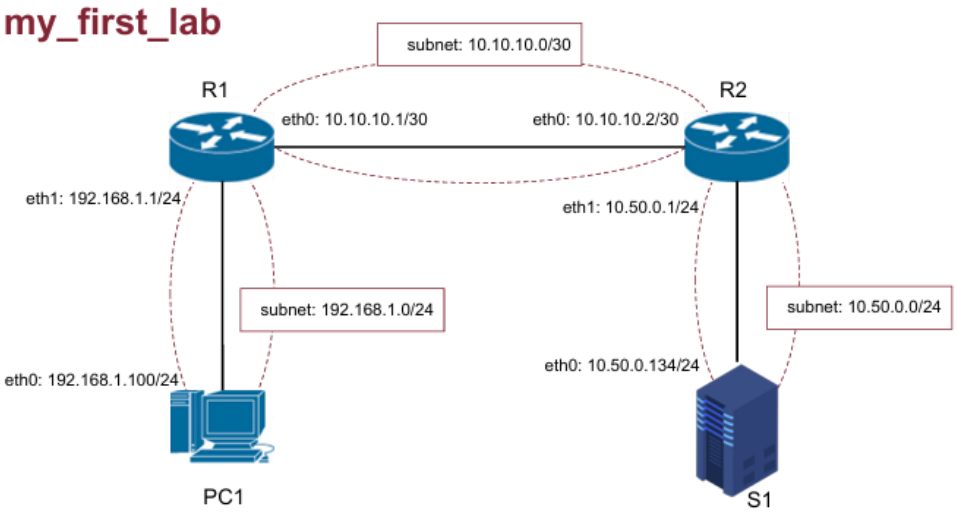
Exercise: Add the route to the previous network that allows pc1 to PING pc2 (and vice versa):



Solution: We should use *ip route add* to tell *pc1*to forward its packets to r1 (then he will know how to send them to *pc2*). Then, we have to use *ip route add* to tell *pc2* how to reach *pc1* (also using r1, then he will know how to send the packets to *pc1*).

[Photo]

Exercise



Make every node able to reach (ping) all other nodes:

1. Create and launch a kathara lab having the topology (collision domains) as in the previous slide
2. Assign static ip addresses as shown via ip address commands
3. Assign necessary routes via ip route commands
4. Ping everybody

Solution: You can find the solved exercise in *Kathara-Lab/Exercises/2-my_first_lab*

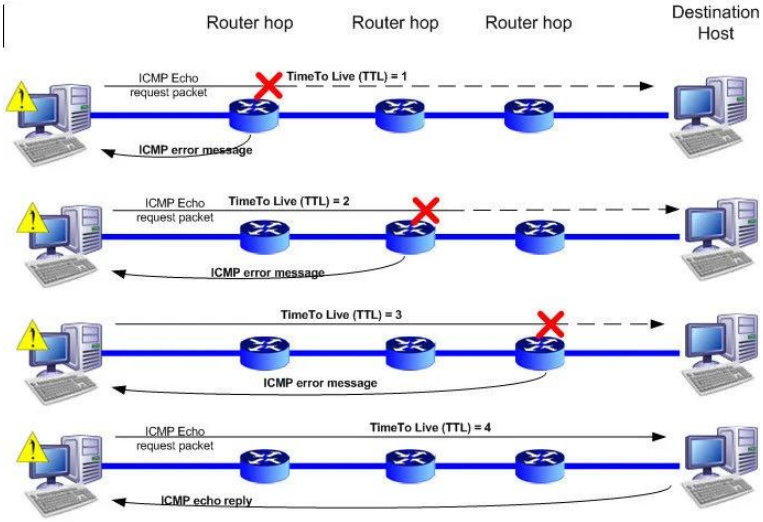
traceroute

Another useful way of testing connection between two hosts is to traceroute the connection via the traceroute command:

```
traceroute a.b.c.d    # finds the number and addresses
                      # of hops to reach a.b.c.d
```

This command exploits the fact that every packet has a **TTL (Time to Live)** field which is X when the packet leaves the interface. Every hop decrease by one the TTL field. When the TTL reaches 0, the host that expired the TTL sends back to the sender an ICMP error packet.

The packet contains the IP address of the host who expired the TTL. By sending packets with TTL starting from 1, traceroute can discover the path taken from the packets to reach a particular destination.



netcat

Netcat allows us to send data over TCP and UDP protocols. In order to use that, we must specify the protocol we want to use, the port we want to use and if we want to send or "listen" on that particular port:

```
nc ip_addr port_num   # connects via TCP to ip_addr
                      # trough port_num via TCP

nc -u ip_addr port_num    # Send UDP datagrams
                          # to ip_addr trough port_num

nc -l -p port_num         # listen for TCP connections on port_num

nc -u -l -p port_num      # listen for UDP datagrams on port_num
```

tcpdump

Linux provides us tools to generate, capture and inspect packets. We can create TCP/UDP packets via the *nc (netcat) command*.

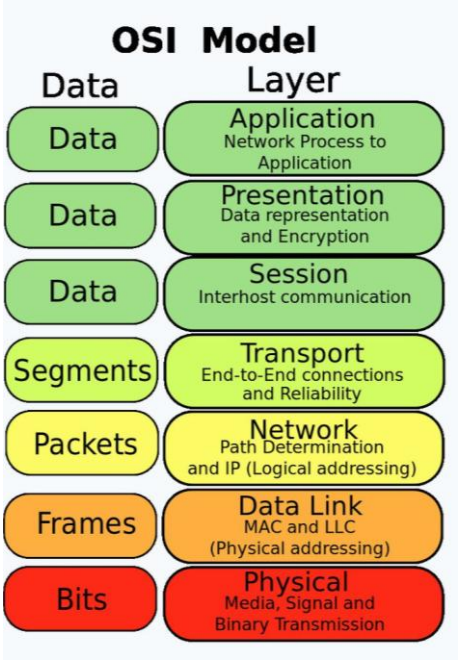
These packets generated can be "captured" via *tcpdump* command and stored to a **"pcap" (packet capture) file.** This file can be later open to be graphically inspected with the **wireshark tool**.

tcpdump allows us to capture packets on a physical interface.

```
tcpdump -i iface_name -w filename.pcap
# capture packets on iface_name and save captured packets on filename.pcap
```

OSI Stack Protocol

The Open Systems Interconnection (OSI) model is the universal framework for computer networking. It breaks down the complex process of transmitting data across a network into distinct, manageable layers.



The Physical Layer (L1): is responsible to define how information is transmitted physically (copper wire, fiber optics, air, etc.) between two entities.

The Data Link Layer (L2): manages, among other things, physical addressing between adjacent nodes in the network. Each network interface has assigned a unique MAC address, used as id. (A single device can have multiple network interfaces!! Each of them has a unique MAC address in the world)

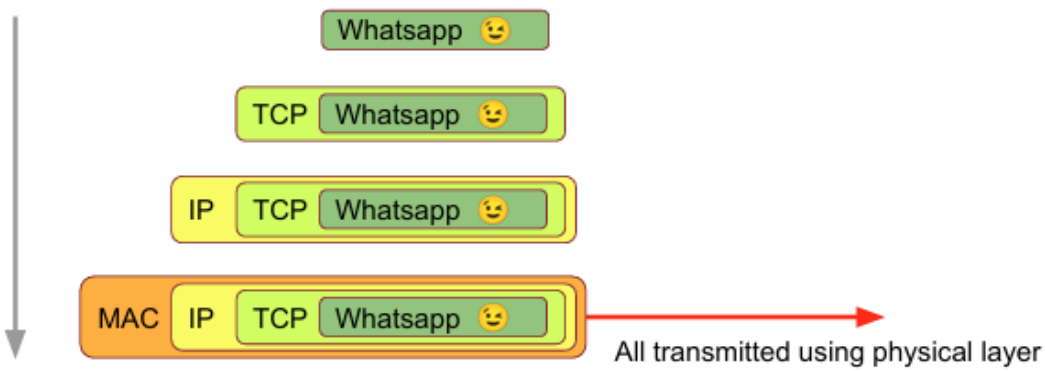
The Network Layer (L3): manages addressing between distant nodes (e.g., nodes in different subnets). Each node has a unique IP address in the subnet.

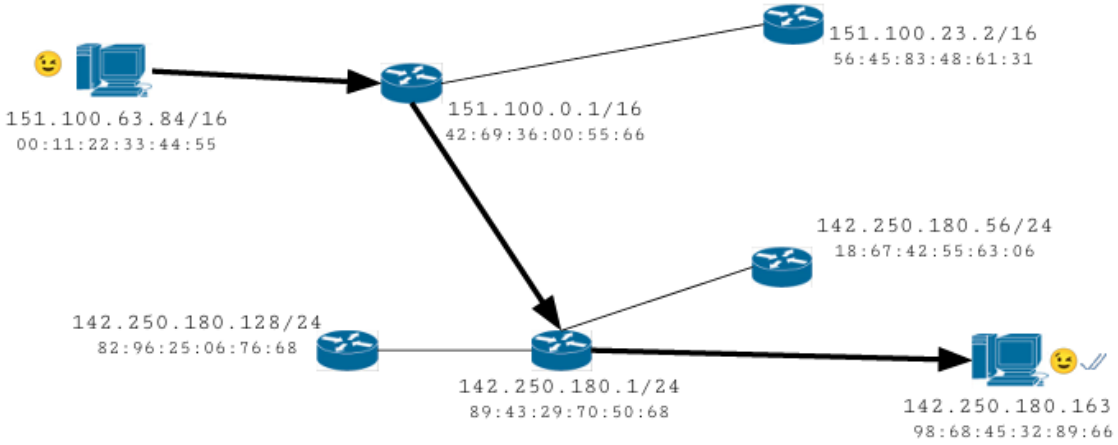
The Transport Layer (L4): manages the end-to-end transport of data (e.g., add ports, multiplex the connections, adds packet acknowledgments, congestion control, prevent out of order packets, etc…). Every well-known application protocol (HTTPS, MQTT, …) runs on top of the transport layer. Netcat uses L4 protocols (TCP or UDP) to send arbitrary data between two hosts.

The Application Layer(s): Application protocols (HTTP, FTP, MQTT) transmit user data end-to-end. In between Transport and Application protocols, there could be protocols such as TLS/SSL to secure application packets.

Packet Encapsulation

When sending a packet using a certain protocol in the stack layer, it gets encapsulated using all the lower layers (and then de-encapsulated at the receiver).





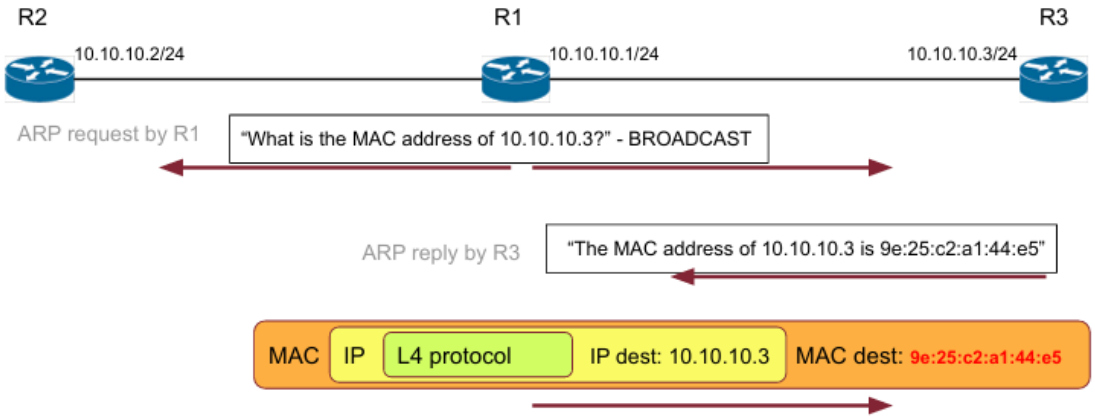
Each node has (at least) one IP address (L3) and one MAC address (L2). But, why?

- **MAC Addresses** uniquely identify a network interface. Each manufacturer manages its own pool of addresses1, and assigns one of them to each network interface. Constant throughout all the lifespan of the device. MAC addresses are used in physical addressing to uniquely identify a node in my neighbors
- **IP addresses** are assigned logically, in such a way to simplify the routing process. Each address belongs to a pool of addresses (subnet). May change over time. IP addresses and subnetting enable routing

In the previous slide, we routed packets from 151.100.63.84 to 142.250.180.163 It required 3 physical addressing, one for each "hop" in the routing.

ARP Basics

The Address Resolution Protocol (ARP) is used to discover the link layer MAC address associated to a given internet layer IP address. Applications are required to specify the destination IP address to which send/receive data, they know nothing about MAC addresses.

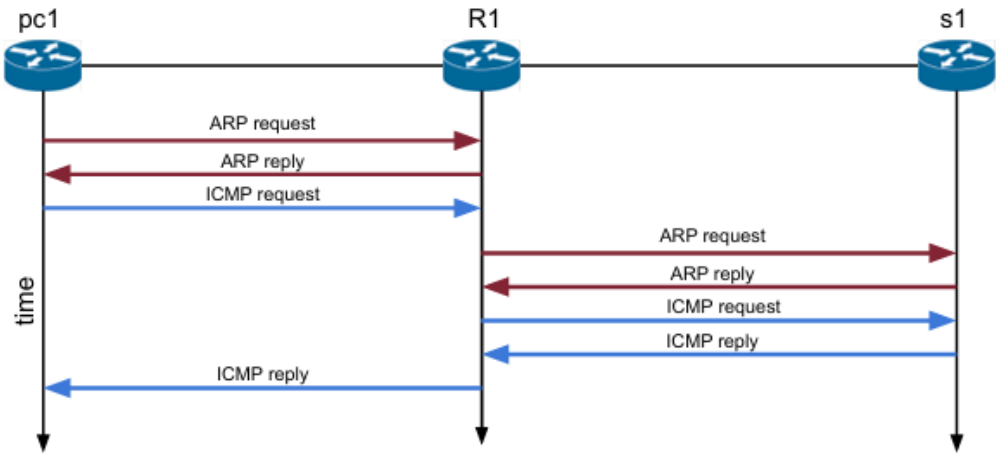


The MAC address collected by ARP are stored in a cache, accessible with *ip neigh* command

```
192.168.1.1 dev wlp1s0 lladdr 14:14:50:7a:d0:bb DELAY
192.168.1.7 dev wlp1s0 lladdr 1e:80:eb:d2:28:e8 REACHABLE
192.168.1.250 dev wlp1s0 lladdr e4:4f:01:00:31:08 REACHABLE
192.168.1.8 dev wlp1s0 lladdr 44:08:0b:7e:68:22 REACHABLE
```

Whenever we want to send a packet to some IP address, we first lookup its MAC address in the ARP cache. If it is already in cache: fine, we can get it quickly! If there isn't we must generate ARP request to get it.

Example: When pc1 pings (ICMP request) s1 for the first time, we observe an ARP request and reply for each hop



Security Consideration - ARP/MAC Spoofing

In theory: MAC addresses are assigned by the manufacturer and are immutable.

In practice, **the linux command to change MAC address is**:

ip link set eth0 address 00:11:22:33:44:55

The problem is that some WiFi and Ethernet networks may use MAC-based authentication. Consequently, ARP alone is not secure. An attacker could "spoof" our MAC address and use it to authenticate.

Solution: network administrator should implement defenses against ARP attacks.

AC addresses are unique values for every network interface of any device → MAC addresses could be used to track a device through space Example: Wi-Fi based localization on smartphones 1. Devices send to their manufacturer the MAC addresses of near Wi-Fi access points 2. There exist databases1 2, even open ones, containing millions of MAC addresses with their position 3. By cross-referencing the MAC addresses sent by the device and the positions of known MAC addresses, you get a rough estimate of the device position

Security Consideration – Tracking and Localization

MAC addresses are unique values for every network interface of any device… So, MAC addresses could be used to track a device through space.

Example: Wi-Fi based localization on smartphones

1. Devices send to their manufacturer the MAC addresses of near Wi-Fi access points
2. There exist databases containing millions of MAC addresses with their position
3. By cross-referencing the MAC addresses sent by the device and the positions of known MAC addresses, you get a rough estimate of the device position

Researchers in have decoded the encrypted diagnostic traffic sent by Android and iOS devices, in particular the traffic exchanged when a device turns on the localization services For example, iPhones [1] with GPS localization enabled logs the MAC address of all nearby devices (connected to the same wifi) and send it to their servers with precise location.

IP Subnetting

An IPv4 address is a 4 bytes-long identifier, expressed in dotted notation, from 0.0.0.0 to 255.255.255.255

- example: 192.168.1.100

A subnet (or prefix) is a logical division of IP addresses. It is a pool of IP addresses. Examples:

- Subnet 192.168.1.0/24 - Contains addresses from 192.168.1.0 to 192.168.1.255
- Subnet 10.10.10.4/31 - Contains addresses from 10.10.10.4 to 10.10.10.5
- Subnet 10.50.0.0/16 - Contains addresses from 10.50.0.0 to 10.50.255.255

The number after the / is the netmask, the number of bits of the subnet

Which IP addresses belong to a subnet? An IP belongs to a subnet if its first `netmask` bits match the subnet.

1.	IP Address decimal	10	8	160	0
2.	Netmask /19	11111111	11111111	11100000	00000000
3.	IP Address binary	00001010	00001000	10100000	00000000

Fixed (Network ID) Variable (Host ID)

How many IP addresses belong to a subnet?

Example: consider the subnet of the previous slide 10.8.160.0/19. We have 19 fixed bits (network ID) and 13 variable bits (host ID). Number of addresses is 213

HOWEVER, we don't assign two addresses:

1. The lowest address 10.8.160.0 indicates the subnet itself
2. The highest address 10.8.191.255 is the broadcast address

So, the total number of assignable addresses is 213 - 2 = 8190

HOWEVER x2, by convention, all the 2 addresses of /31 subnets are assignable. E.g. in a subnet 10.0.0.0/31 both 10.0.0.0 and 10.0.0.1 are valid

To which subnet does an IP belong?

In other words, given an IP address and a netmask, how to find its subnet? You can obtain the subnet by applying the bitwise AND of the address and the netmask Example: considering an IP address 10.8.162.52 and knowing that its subnet has netmask /19, which is its subnet? Steps:

1.	IP Address decimal	10	8	162	52
2.	IP Address to binary	00001010	00001000	10100010	00110100
3.	Netmask* /19	11111111	11111111	11100000	00000000
4.	Subnet binary	00001010	00001000	10100000	00000000
5.	Subnet to decimal	10	8	160	0

IP Routing

Def. **Routing is the process of forwarding packets towards its final destination.**



The IP Subnets we have just seen are fundamental because they enable routing! The nodes doing routing are routers. For each incoming packet they look up in a Routing Table and decide which is the next hop (another router or the final node). Then, they simply send the packet on right interface to the next hop.

Routing Tables

We can get the routing table of a certain router using the command *ip route show*. Generally, each entry in the routing table indicates how to reach a certain subnet, not a specific IP address.

Destination Subnet	Interface	Metric
Default	eth0, via 10.50.0.1	100
10.2.128.0/18	wlp1s0	600
10.50.0.0/16	eth0	100
169.254.0.0/16	eth0	1000

Note that in the routing table, we store the interface to which forward a packet. But we did not specify an interface! next_hop_ip is itself routed recursively to find its target interface!

Forwarding Decision

When a router wants to send a message to the IP Address A, for each entry in the routing table, the router checks if A belongs in that entry. At the end:

1. **If only one subnet matches**: forward the packet there
2. **If more than one subnet matches**: Longest prefix match
3. **If no subnet matches**: Is there a “default” entry?
    - **Yes**: forward it using the default gateway
    - **No**: Network Unreachable error. Translates to “I don’t know how to reach that subnet”

Esempio: Supponiamo la seguente tabella di routing:

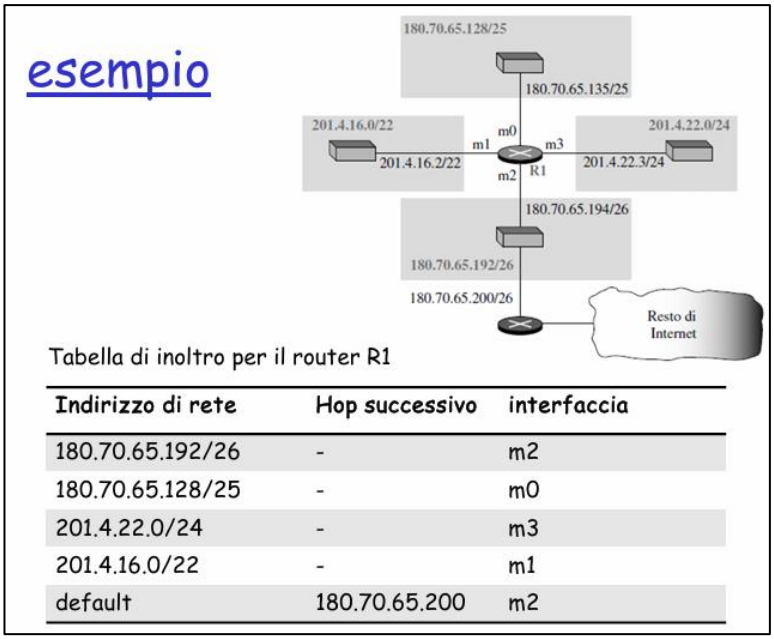


Tabella di inoltro per il router R1

Indirizzo di rete	Hop successivo	interfaccia
180.70.65.192/26	-	m2
180.70.65.128/25	-	m0
201.4.22.0/24	-	m3
201.4.16.0/22	-	m1
default	180.70.65.200	m2

Se dobbiamo inoltrare il datagramma, con indirizzo di destinazione 180.70.65.140 = 10110100 01000110 01000001 10001100, il router esegue i seguenti passaggi:

1. Confronta con la prima maschera (/26) ed avremo:

   10110100 01000110 01000001 10001100 (indirizzo destinazione)

   10110100 01000110 01000001 11000000 (maschera presente nella tabella)

   I due numeri non coincidono. Dobbiamo quindi controllare la entry successiva

2. Confrontiamo con la seconda maschera (/25) ed avremo:

   10110100 01000110 01000001 10001100 (indirizzo destinazione)

   10110100 01000110 01000001 10000000 (maschera presente nella tabella)

   I due numeri combaciano alla perfezione. Questa è la entry giusta. Possiamo quindi inoltrare il pacchetto all'interfaccia m0

Exercise: Address to route: **151.100.25.63** Routing table contains the following entries:

A) 151.64.0.0/10
B) 151.96.0.0/11
C) 151.0.0.0/8
D) 151.112.0.0/12
E) 151.104.25.0/28

Which one would you choose?

Exercise – unreachable_destinations

In this lab we try to observe the network behavior in case of errors. Depending on the nature of the error we observe a different behavior, useful when debugging. Launch unreachable_destinations lab, already configured, available now in the shared drive. Then:

1. From pc1 try to ping an unreachable local destination, e.g. 192.168.1.7, in the same subnet of pc1. Capture on eth0 pc1 (e.g., tcpdump -i et0 -w /shared/cap.pcap)
2. From pc1 try to ping an unreachable remote destination, e.g. 10.8.0.20, in the same subnet of s1. For this, capture packets on interface eth0 and eth1 of R2.
3. From pc1 try to ping a non-existent remote destination, e.g. 172.16.0.1. Capture on eth0 pc1
4. Observe with wireshark which ARP packets are exchanged in the various collision domains. Which errors do you observe?

Tip: You will need two terminals for pc1 (one pinging and another one sniffing). You can open a new terminal typing *kithara connect pc1*.

Solution: **TO DO**

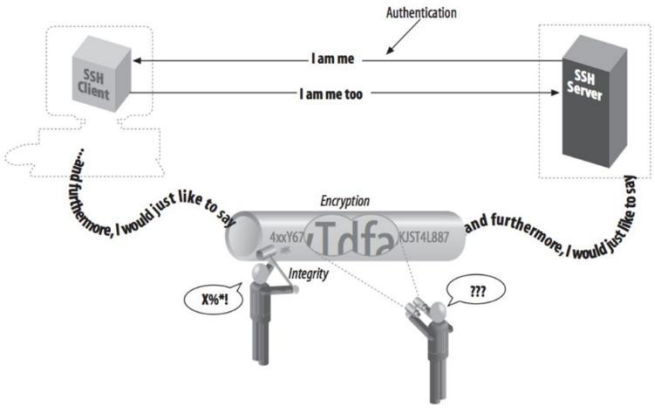
SSH – Secure SHell

In real life, physical access to network nodes is not always an option. Often, we need to reconfigure network nodes remotely, usually with a terminal. Historically, the first protocol that allowed such things was telnet.

Telnet has been now deprecated since it does not use any encryption on the sent data!

It is now replaced with SSH. SSH (SSH-2) is a protocol that allows remote management over a secured channel. Is a client-server protocol. The main features that SSH provides are:

- Authentication
- Encryption
- Integrity



Every Linux distro has a ssh client installed called OpenSSH client. On the other hand, the server daemon, sshd, needs to be manually installed. The configuration file of the server is */etc/ssh/sshd_config* while for the client is */etc/ssh/ssh_config*.

To start a connection with a server:

```
$ ssh username@host_ip_addr
# starts ssh session with the host specified by
# host_ip_addr logging in as username
```

You **logout with CTRL+D**. By default, SSH runs on TCP port 22. We can change the port through the command:

```
$ ssh username@host_ip_addr -p 12345
```

Creating a User on the Server

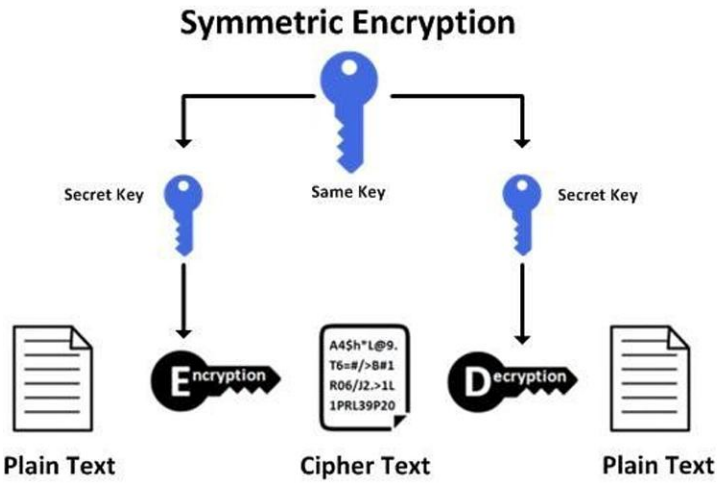
In order to connect via SSH to a user, the user should be present on the server and should have a password:

1. Restarts the networking and the ssh daemon
2. Creates the user ssh_user with its home folder
3. Assigns the password “ilovessh” to ssh_user. (Without a password we can’t perform password authentication)

```
/etc/init.d/networking restart
/etc/init.d/ssh restart
useradd ssh_user -m
echo -e 'ilovessh\nilovessh\n' | passwd ssh_user
```

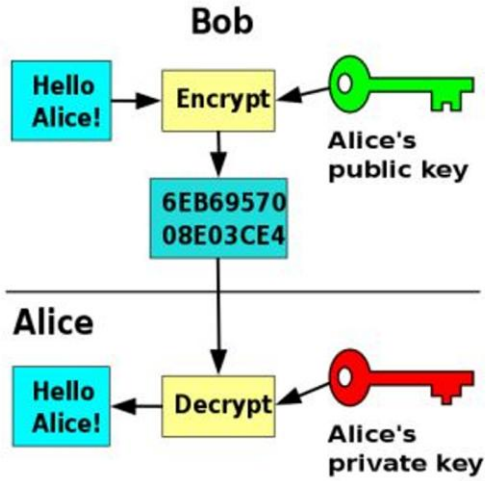
Symmetric-Key Cryptography

In Symmetric cryptography, the same key is used for encryption and decryption Most widely used symmetric key algorithms include Twofish, AES, Blowfish, …



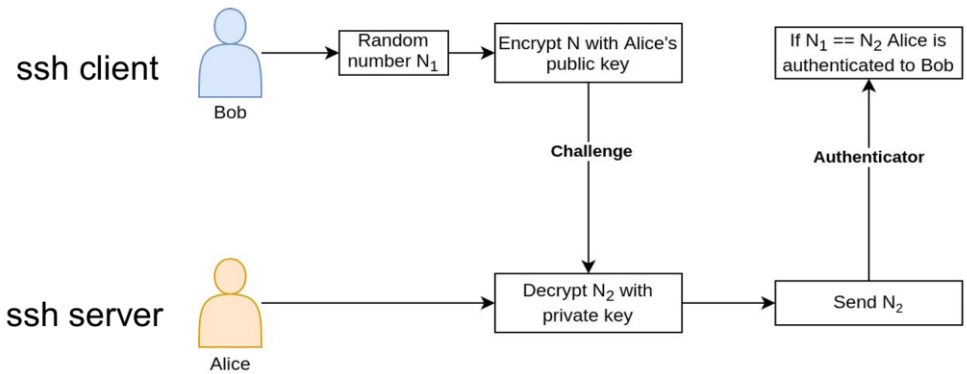
Asymmetric-Key Cryptography

In Asymmetric cryptography we have two keys (often called a key-pair): data is encrypted with one key and it is decrypted with the other key! Idea: we can share with everyone the key needed for encryption (public key) while keeping the decryption key as a secret (private key).

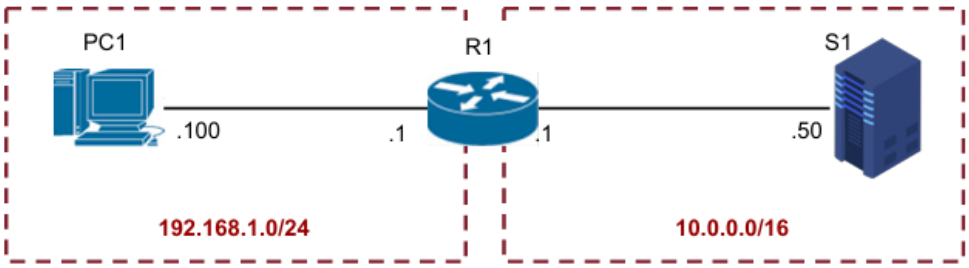


In practice, asymmetric-key ciphers are too slow (compared to the symmetric one) to be used to chipper text. On the other hand, Asymmetric-Key is much more secure and used for **authentication**.

Challenge-Response Authentication: Bob sends to Alice a challenge that can be solved only by whom has the private key. At the end Alice is authenticated to Bob. *(More complex methods may be involved but the concept is the same)*



Exercise: lab_ssh



First of all, we have to create a user:

```
root@s1:/# useradd -m ssh_user1
root@s1:/# passwd ssh_user1
New password:
Retype new password:
passwd: password updated successfully
root@s1:/#
```

Connect to the server:

```
root@pc1:/# ssh mynewuser@server
ssh: Could not resolve hostname serve
```

Dobbiamo prima dare l'alias all'ip del server, ma non ricordo quale è l'ip 😊

SSH Protocol Summary

Encrypted connection setup

1. The client opens a TCP on (by default) port 22. They both disclose the SSH protocol versions they support
2. They both provide a list of accepted cipher methods
3. The client selects a cipher method and negotiate a *session key*
   a. From now on the rest of the communication is encrypted symmetrically using the *session key*. It provides encryption.

Encrypted connection after

Server authentication

4. The server identifies itself to the client: it sends to the client its *host key*
5. The client sends a challenge to the server
6. The server replies with an authenticator
7. The client checks the authenticator. If all went well the server is authenticated to the client.

Client authentication

8. The client now proceeds to authenticate to the server, either with *password* authentication or with *public key* authentication

Server Authentication

The server sends his public key (named host key) to the client. The client is now asked to trust or not the host key fingerprint.

```
iMac-di-Marco:~ marcospaziani$ ssh marcos@stud.netgroup.uniroma2.it
The authenticity of host 'stud.netgroup.uniroma2.it (160.80.221.14)' can't be established.
ECDSA key fingerprint is SHA256:UKRm/pfFevBVndWv6nAjdOpueHhFacxD5XsiswY3Q20.
Are you sure you want to continue connecting (yes/no)?
```

If yes, the client sends challenge based on the host key. The server replies with authenticator. If the authentication process is successful the server is authenticated to the client.

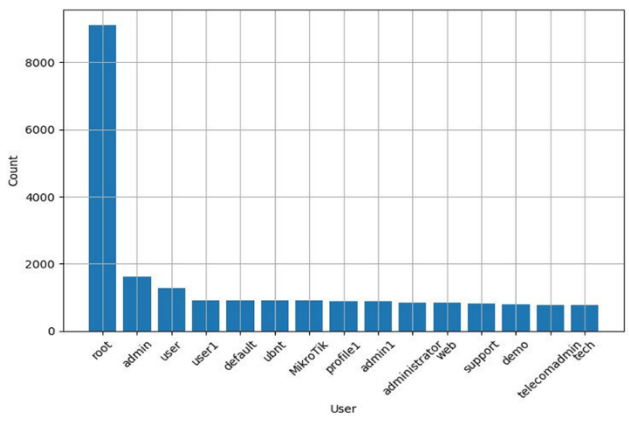
Client Authentication

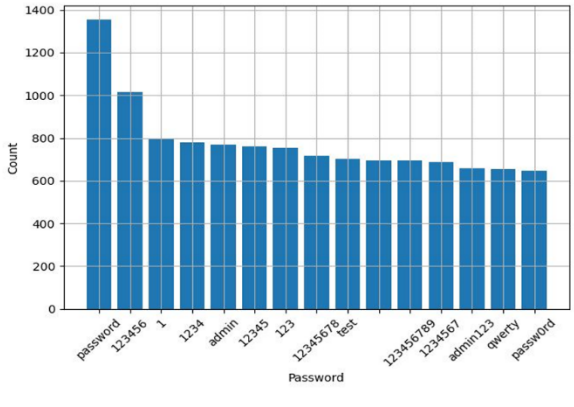
For our purposes, we consider two methods:

- **password authentication**: With password authentication the client sends the password as-is to the server. Remember that we established an encrypted connection before, therefore a man-in-the-middle cannot spoof the password.

  However… what happens if the server itself is malicious? With performing password authentication you just gifted your password to the attacker!!

NOTE: On a side note, the internet is full of bots trying to access SSH servers. Below I reported the most common username and password tentatives on a SSH server with password authentication enabled in a time window of ~72 hours





- **public-key authentication:** the client must possess a public-private key pair. A classic challenge-response authentication takes place:
    1. The server sends a challenge to the client
    2. The client sends back to the server the authenticator
    3. If all goes well the client is authenticated

Enabling Public-Key Authentication

Steps:

1. **Generate a new public-private key pair on the client**: We want to make a step forward and use asymmetric auth. In the following example pc1 will be the client and pc2 the server. First of all, we have to generate a key pair on the client.

```
$ ssh-keygen
```

A hidden folder inside the home folder of the user, a directory called .ssh is created. Inside that there are 3 files:

```
$ known_host    #Database of the trusted fingerprints
$ id_rsa        #private key generated using RSA
$ id_rsa.pub    #public key generated using RSA
```

We now have to transfer the public key of the client to the list of trusted client keys of the server

2. **Transfer the public key from the client to the server**: We can transfer files between remote hosts using “scp” (Secure CoPy).

```
$ scp local_path user@host_ip:/remote_path
# transfer file at local_path to remote_path over SSH

$ scp user@host_ip:/remote_path local_path
# transfer file at remote_path to local_path over SSH
```

scp is based itself on ssh: it connects to user@host_ip and copies the files. On the client (pc1) we can use the following command to copy its public key to the server (pc2):

```
$ scp .ssh/id_rsa.pub ssh_user@s1:/home/ssh_user/client_key.pub
```

3. **Authorize the new key on the server (Optional):** We have to put the public key of the client into the trusted clients “database” in the server. The “database” is the file */home/ssh_user/.ssh/authorized_keys* which we have to create:

```
$ mkdir .ssh                        #create .ssh folder
$ touch .ssh/authorized_keys        #create authorized_keys file
# Set correct permissions
$ chown ssh_user:ssh_user /home/ssh_user/.ssh
$ chmod 700 .ssh
$ chown ssh_user:ssh_user /home/ssh_user/.ssh/authorized_keys
$ chmod 600 .ssh/authorized_keys
```

After done all of this we can copy the public key of the client to the list of trusted keys of the server, by appending the public key to the authorized_keys file:

```
$ cat client_key.pub >> .ssh/authorized_keys
```

4. **Remove the possibility of Password Authentication**: Once public key authentication has been set up, one should remove the option to perform password authentication (must be done on server side). In the server, there is a file /etc/ssh/sshd_config containing configuration options of the ssh server

   PasswordAuthentication no

   And after that restart the ssh daemon via:

   ```
   $ /etc/init.d/ssh restart
   ```

Tip: The steps 2 and 3 can be done together using the command:

```
$ ssh-copy-id ssh_user@pc2
```

Behing the scenes, this command creates a ssh connection to ***ssh_user@pc2***, copies the key and add it to the *autorizhed_keys*. **However, this method only works if you can already access *ssh_user@pc2* via ssh!**

Exercise: lab_ssh_live

This will work only during the current lecture.

On my personal server1 (*ni-test.ddns.net*). I've given to each one of the enrolled in the course a user:

- **Username**: user<MATRICOLA>
- **Password**: your surname

Task: connect to the server and setup a public-key authentication for your user

- **Step 1**: create the key

```
(base) PS C:\Users\39324\Desktop> ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\39324/.ssh/id_ed25519):
Created directory 'C:\\Users\\39324/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\39324/.ssh/id_ed25519
Your public key has been saved in C:\Users\39324/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:iE15MKaoxMHRdaoLN/0ed2mSwMOFYdRWK1Fw4MTjeYI 39324@PC_Francesco
The key's randomart image is:
+--[ED25519 256]--+
|.oo ..==o==+     |
|...o o++==o .    |
| o. ..o.=+o.     |
|..  o+oE.+..     |
|.. +..o=S o      |
|  o o . o . .    |
|   .   o + +     |
|      . o +      |
|       .         |
+----[SHA256]-----+
```

NOTE: We must memorize the key path to send it to the server!!

- **Step 2**: Transfer the public key from the client to the server

```
(base) PS C:\Users\39324\Desktop> scp C:\Users\39324/.ssh/id_ed25519.pub user2108040
        1 file copiati.
```

- **Step 3**: Connect to the server

```
(base) PS C:\Users\39324\Desktop> ssh user2108040@ni-test.ddns.net
```

- **Step 4**: Authorize the new key on the server

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
cat ~/temp_key.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
rm ~/temp_key.pub
exit
```

NOTE: Steps and 4 can be done together with the command:

```
ssh-copy-id user2108040@ni-test.ddns.net
```

Then, we only have to connect to the server

SSH Port Forwarding

In the context of SSH, Port Forwarding redirects packets inside an existing SSH connection. Typical use cases include:

- Running a public web service inside a LAN
- Bypassing firewalls filtering specific ports and/or IP ranges
- Permitting SSH access to a host on the private LAN from the Internet

SSH gives you the possibility to perform **local** or **remote** port forwarding.

Local Port Forwarding

Scenario1: Suppose you are abroad (e.g. Germany) and the server you wish to connect has some regional restrictions (e.g. Netflix) that does not allow you to visit a very important page that you could actually access in Italy.

How to solve that? → SSH Local Port forwarding

The only thing we need is that our home PC has an SSH server running and that we can access with a user (not necessarily with asymmetric auth).

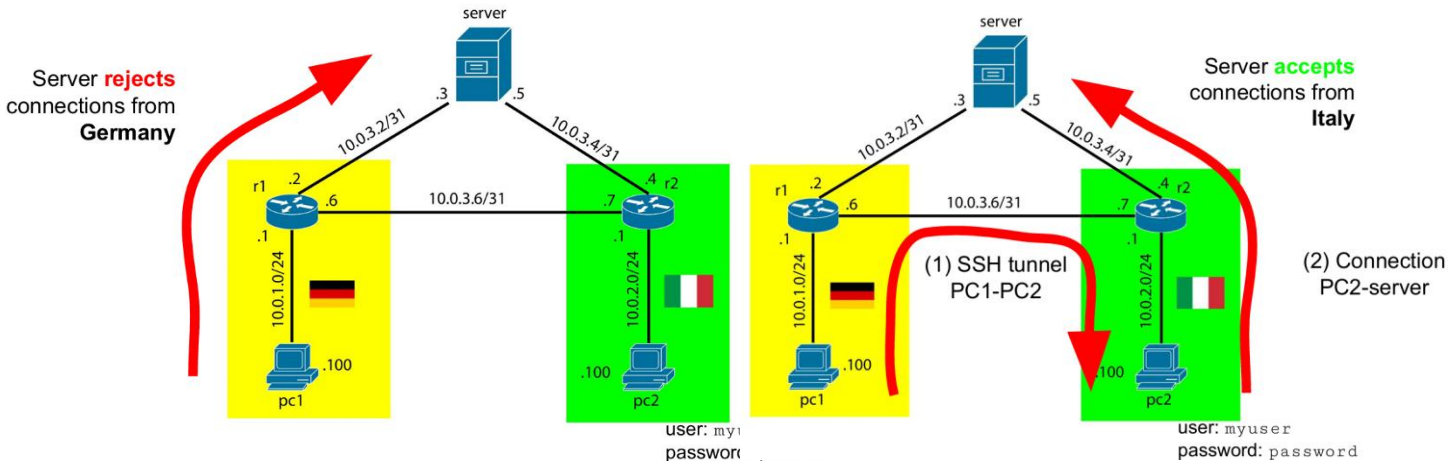
1. On server runs an echo server on TCP port 8080.
2. The server has a firewall blocking all requests coming from Germany subnet: if we launch netcat to connect to the server from pc1 we obtain no response.
3. We now want to redirect all the traffic directed to the 8080 port of server through an SSH tunnel over 9000 from pc1 to pc2:

   pc1: $ ssh -NL -v 9000:10.0.3.5:8080 user@10.0.2.100

4. Now on pc1 (on another terminal) we can connect to the server with:

   pc1: $ nc localhost 9000

5. Every packet sent to localhost:9000 is redirected through the SHH connection and then sent from the SSH host to 10.0.3.5:8080



In general, we have:

ssh -NL local_p:dest_ip_addr:dest_p user@tunnel_ip

Where -N no shell and -L local port forwarding

NOTE: if you see “bind: Cannot assign requested address” it’s not an error, but just a warning. Can be disabled by passing “-4” parameter to ssh command)

Scenaro2: I want to play some multiplayer games instead of following the Network Infrastructures lessons.

The university firewall is blocking the port on which the game runs :(

We can solve it thanks to Port Forwarding!! Most of the applications that use internet messaging rely on TCP/UDP. This includes HTTP(s), emails, and games.

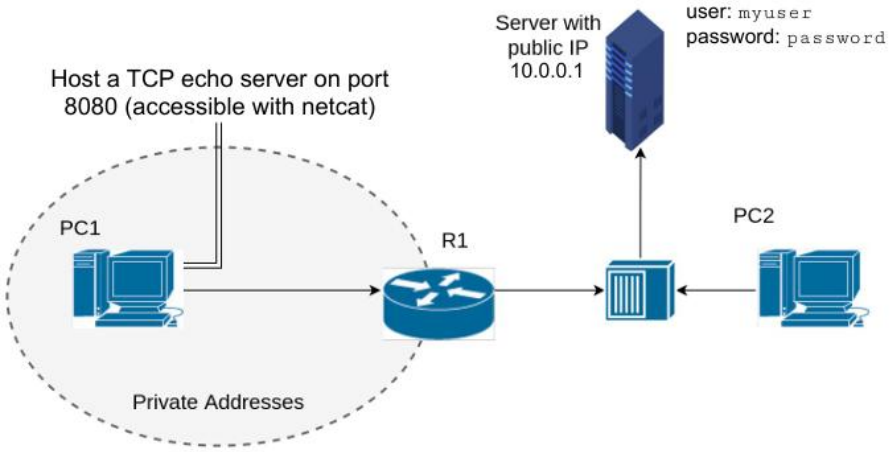
Remote Port Forwarding

The concept of remote port forwarding is the same of local port forwarding: opens a listening port on the remote host (instead of a local port of local port forwarding)

Useful especially to bypass NAT and/or private addresses restrictions:

A host h1 cannot initiate a connection to h2 if h2 has not a public address

NOTE: Private addresses are used everywhere (e.g. your home wifi, Sapienza LAN, etc.)



Scenario: Suppose I am on a LAN without a public IP address (e.g. at home). On my PC it's running some web service I am developing (e.g. hosting a web page) and I want people from outside the LAN to connect to this service. From PC1 we can:

1. Initiate a SSH connection
2. Tell the server to open a listening port, e.g. 9000
3. Tell the server to tunnel every packet destined to its port 9000 to PC1:8000

pc1: $ ssh -NR 9000:localhost:8080 myuser@10.0.0.1

4. Forwarding

ssh -NR remote_p:dest_ip_addr:dest_p user@tunnel_ip

By default, the forwarded port on the server listens only on localhost. If we want it to listen on all the available interfaces in the file sshd_config on the server there should be a line: GatewayPorts yes

